Last modified on

Sports Data Soccer API - Match Stats Feed (MA2)

Get detailed match statistics for teams and each individual player, including passes, shots, crosses, tackles and more.

Overview

The Soccer Match Stats feed contains detailed match statistics for teams and each individual player - this includes Passes, Shots, Crosses, Tackles and much more. You can see a complete list of match statistic types in this guide.

How Often Should I Poll The Feed?

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Make sure that you do not exceed the rate confirmed with us - this can vary depending on whether you make requests as a B2B or B2C outlet - if you need more information on this, contact your Stats Perform representative.

UPDATED	POLLING FREQUENCY							
	TIER 15	TIER 12+14	TIER 4	TIER 11+13	TIER 8+9+10	TIER 6+7	TIER 5	TIER 1+2+3
Updated on stops in play during the game, including for the following types of events: goal, shot at goal, assist, free kick, offside given, cards, corner won, corner lost, substitution, lineup	See Tier 13	Post-match	n/a	No more frequently than one URL poll every 30 seconds for range: <KO - 1h; KO + 4h> Up to 1-hour interval for range: <KO + 4h; KO + 8h> Up to 24-hour interval 8h+ of match ending, lasting a maximum of 7 days		n/a	n/a	n/a

Note: The feed should update approximately every 90 seconds and will be pushed out automatically in event of key actions as above. However, please note:

- There will always be a gap of at least 30 seconds between files unless a goal is scored - in which case, a file will be produced immediately.
- There may be a gap of several minutes between feeds if no key events take place.

- Please consider using the **Delta parameter** feature before integration as this can hugely improve feed performance.

Use Deltas To Get Updates

Get updated data and make streamlined requests using the Delta parameter for a list of live matches

Where your call will return a list of live matches (does not specify a fixture UUID to return only one match), we recommend that you use the Delta (`_dlt`) parameter in your requests, so that you can programmatically get the latest assets since a specified date/time. In MA2 the specified time should not be beyond 120 seconds. This is valid when request **a list of matches** (not a single match) that are **live/in progress**; therefore, the call must be valid and contain `status=playing` and `_dlt` (with a valid `=`*dateTime* value).

Using Deltas could significantly benefit and streamline your use of the SDAPI, reduce your polling overheads, and improve latency and overall performance. This allows you to return and ingest only the data that has changed in a specific time period from the feed (instead of receiving all data and checking each individual match). You can use the Delta parameter with existing queries, such as to get detailed and people data. See Query Parameters And Example Request Parameters for an example.

Match Stats (MA2) Guide

USAGE	FEED
GET **match stats** for a fixture by fixture UUID (path parameter)	`/soccerdata/matchstats/{`
GET **match stats** for one or more fixtures by UUID (`fx` parameter - see the Query parameters below)	`/soccerdata/matchstats/{ {queryParameters}`

If your request URL is as follows (does not contain the fixture UUID query criteria), the feed returns an error: `/soccerdata/matchstats/{outletAuthKey}`

How can I get fixture and other entity UUIDs?
Use the Tournament Calendar (OT2) or Fixtures and Results (MA1) feeds to get fixture metadata, including UUIDs for each fixture and its linked contestants, competition, tournament calendar and other entities.

Query Parameters And Example Request Parameters

This feed also makes use of the Global query parameters, and you can combine them with the following query parameters (some of which can be used in conjunction with each other). However, you must query the `/matchstats` feed by a fixture (match) UUID.

Legacy IDs: You can use the `fx` query parameter to filter by legacy IDs (only one ID is supported - you cannot specify a comma-separated list of IDs).

How to use query parameters and make successful requests:
In our guides, we use HTTPS syntax for our example `GET` requests (rather than cURL, for example). All URLs are case-sensitive - this means that URL strings must contain the correct case and operators. You MUST format your

requests correctly and include certain information; otherwise, an error will be returned. You can find information about the API, including the base URL (`{protocol}://{requestDomain}/{feedResource}`), in the main guide and API Overview and Authorisation guide.

- Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid `{outletAuthKey}` this must be after the Outlet Authentication Key (and endpoint, if applicable). for `_rt={mode}` (operating mode) and `_fmt={dataFormat}` (response format). If required, it should also include an entity UUID (such as a fixture/match UUID).
- Begin the query parameter string with `?` (question mark) - this must be after the Outlet Authentication Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by the `&` (ampersand) operator. Values must be separated by the correct operator, for example `&` (multiple values, 'AND'), or - if a parameter supports it - `,` (multiple values, 'OR') or `!` ('NOT').
- Use the required `_` (underscore) for any parameters listed with this prefix.
- Remove any placeholder value curly braces `{}` from your calls (we include these as an example only).
- If using the JSONP format (`_fmt=jsonp`) you MUST use it in combination with the callback parameter (`_clbk`) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly-generated function names - you must make sure that these are overridden and define a fixed function name instead.

Query parameter	Value and description
`_dlt` + `status`	Programmatically get the **latest data updates since a particular date/time** (created, updated and published) for **matches currently in progress** (`status=playing`). Note: The `_dlt` parameter is not valid when the request specifies a single fixture ID in the call Value: Replace `dateTime` with `xsd:dateTime` (with timezone) or `milliseconds` (since the epoch) to specify the date and time. The format of the date/time for XSD UTC/GMT is: `YYYY-MM-DDThh:mm:ssZ` `GET https://api.performfeeds.com/soccerdata/matchstats/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&_dlt=dateTime&status={statusType}` **Note:** The default behaviour is all types (equivalent to `status=all`). You can pass only one value to the `status` parameter. Available values: `Fixture` , `Played` , `Playing` , `Cancelled` , `Postponed` , `Suspended` , `All` (default). In the Soccer API, the values for `status` are not case-sensitive - for example, both `status=fixture` and `status=Fixture` are valid. The specified time should not be beyond **120** seconds. We also recommend that you take a look at our Use Deltas To Get Updates.
`fx`	GET match stats and information about one fixture or multiple fixtures. • To return one fixture: Pass a fixture UUID or legacy Opta match ID to the `fx` parameter. • To return multiple fixtures: Pass the fixture UUIDs in a comma-separated list to the `fx` parameter. For the B2B outlet request type (`_rt=b)`) you can pass up to 50 UUIDs, while a B2C request (`_rt=c`) allows up to 20 UUIDs. Multiple legacy Opta and Opta Core match IDs are not supported.

Query parameter	Value and description
	`GET https://api.performfeeds.com/soccerdata/matchstats/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&`**`fx={fixtureUuid}`**
`fixture/match UUID`	GET match stats and information about one **fixture** by passing the fixture/match UUID as a path parameter (the asset/resource) in the URL. `GET https://api.performfeeds.com/soccerdata/matchstats/{outletAuthKey}/`**`{fixtureUuid}`**`?_rt={mode}&_fmt={dataFormat}`
`detailed`	GET an extended list of **(detailed) statistics**. Pass a value of `yes` or `no` (default) to the `detailed` parameter. `GET https://api.performfeeds.com/soccerdata/matchstats/{outletAuthKey}/{fixtureUuid}?_rt={mode}&_fmt={dataFormat}&`**`detailed={yes|no}`** GET an extended list of (detailed) statistics. Pass a value `fallback` to the `detailed` parameter. `GET https://api.performfeeds.com/soccerdata/matchstats/{outletAuthKey}/{fixtureUuid}?_rt={mode}&_fmt={dataFormat}&`**`detailed={fallback}`** • When requested with `detailed=fallback,` output should contain detailed feed, only if subscribed to detailed version. • When requested with `detailed=fallback,` output should return basic feed, only if subscribed to basic version. • When requested with `detailed=fallback,` output should return 403 error, if not subscribed to both basic or detailed version. • When requested with `detailed=fallback` / `detailed=yes` + `people=no` output will return player stats. **Note:** The fallback parameter is available only for a single match. When used with multiple matches, you will receive error 10203 (Incorrect combination of parameters).
`people`	GET statistics for all people within a team (default) or teams only. Pass a value of `yes` (default, returns all people) or `no` (teams only, not people) to the `people` parameter. `GET https://api.performfeeds.com/soccerdata/matchstats/{outletAuthKey}/{fixtureUuid}?_rt={mode}&_fmt={dataFormat}&`**`people={yes|no}`**
`_lcl=en-op`	GET assets with the **Opta namings in English** in the response (instead of Opta Core namings). When the call does not include the `_lcl=en-op` parameter-value, the feeds will continue to work as they do currently; one example is that the response will be in the default language of English unless the call contains the `_lcl` parameter and a locale code value to request the response in a different locale/language. **Note:** You can use this parameter in combination with other valid parameters and values in the request. However, you cannot use it in combination with a locale/language code and `_lcl` parameter.

Query parameter	Value and description
	`GET https://api.performfeeds.com/soccerdata/matchstats/{outletAuthKey}/{fixtureUuid}?_rt={mode}&_fmt={dataFormat}&_lcl=en-op&{queryParameters}`
`_pgNm` + `_pgSz`	GET data only in the specified page size and/or page number. All results are returned in pages and these parameters help limit the page size in order to optimise the efficiency of the SDAPI. • `_pgNm` - Specify the `{pageNumber}` to return - indexed from `1`. If this value is not provided, then page 1 is returned by default. • `_pgSz` - Specify the size of the page to return - the number of match definitions in a page. Valid `{pageSize}` values depend on the operating mode of an outlet. For B2B requests, the maximum value is `1000` (default is 20). For B2C requests, the maximum value is `50` (default is 20). `GET https://api.performfeeds.com/soccerdata/matchstats/{outletAuthKey}/{fixtureUuid}?_rt={mode}&_fmt={dataFormat}&_pgSz={pageSize}&_pgNm={pageNumber}&{queryParameters}`
`_fld`	GET only the subsets of data fields that you specify, instead of all points, by passing specific values to the `_fld` parameter in the URL. This also reduces the size of the feed, loading speed, and only pulls back the relevant, specified data. All existing MA2 parameters can be used in combination with `_fld`. Values of `_fld` parameter are based on position of fields in the XML. They are shortcuts of the relevant path to the field; therefore, once you take a look, you should be able to use the following logic to get the values: • first **two** letters of every word of a field name - eg `main.da` = `matchInfo` + `date` • first **three** letters of a word for one-word parts of path - eg `main.con.con.id` = `matchInfo` + `contestant` + `contestant` + `id` • first **two** letters of each word for multi-word parts of the path - eg `main.da` = `matchInfo` + `date` Example: The call returns `matchInfo` element (`main`), match date and ID (`main.da`), each contestant ID and position (`main.con.con.id`), competition, and country ID (`main.com.cou.id`): `GET https://api.performfeeds.com/soccerdata/matchstats/{outletAuthKey}/{fixtureUuid}?_rt={mode}&_fmt={dataFormat}&_fld=main.da,main.con.con.id,main.com.cou.id`

> Do not supply both the `_lcl` (with a locale/language code value) and `_lcl=en-op` in the same call - this is not supported; otherwise, the feed will return an error and code `10203` (Ambiguous/Invalid request parameters) in the response due to a duplicated `_lcl` parameter.

Sample Calls

These sample calls let you see an example URL and the response - click on the links below (hyperlinks open in a new window).

Each response provides data points about the match, including any available team (contestant) and player stats. The feed only returns basic stats by default - you will only receive both basic and detailed stats on request, by passing this parameter-value in the call: `detailed=yes`. You can get an explanation of the data contained within the response in Feed Output, Team/Contestant Stats and Player Stats.

- XML - basic
- JSON - basic
- XML - detailed
- JSON - detailed
- XML - detailed & VAR review event

Example (_dlt):

```
https://api.performfeeds.com/soccerdata/matchstats/1vmmaetzoxkgg1qf6pkpfmku0k/bsu6pjne1eqz2hs8r3685vbhl?_
fmt=xml&_rt=b&_dlt=2012-06-07T07:00:00%2000:00
```

The `_dlt` parameter is intended to be used in applications that periodically poll the feed for updates, and should be given a value that corresponds to the date/time of the previous poll to the same feed. By integrating the `_dlt` parameter in your code, you can automate polling, retrieving and publishing updated assets from the feed.

> **Important:**
> • Make sure that there is no gap in the frequency between requests for new assets and the Delta time period value used to request updated assets from; otherwise, you will not capture any updates/changes made to assets, including those you have previously retrieved.
> • In the case of the Match Stats feed, you should only use the `_dlt` parameter to return a **list of matches that are currently in progress** (not a single match).
> • When delta parameter is used in MA2, It will fetch data from the last 120 seconds. If you try to fetch data beyond 120 seconds from the current time, you will receive an error code 10213 - Invalid delta requested.

Feed Output

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

> - The root element can be `matchStats` or `matchInfo` - see the table below.
> - In the scenario where the match has not yet been played (for example, it is a scheduled fixture to be played in the future), not all attributes listed here will be present and we may provide placeholder values for some attributes. For example, the scores, winner, contestant name, position, and ID may not be known (the contestant `id` attribute may also not be returned). However, the feed will be updated with this information once it is available.

▸<matchStats> (XML only)

▸<matchInfo>

▸<description>

▸<sport>

▸<ruleset>

▸<competition>

▸<country>

▸<tournamentCalendar>

▸<stage>

▸<series>

▸<contestants> (XML only)

▸<contestant>

▸<country>

▸<venue>

▸<liveData>

▸<matchDetails>

▸<periods> (XML only)

▸<period>

▸<suspensions>

▸<suspension>

▸<scores>

▸<ht>

▸<ft>

▸<et>

▸<pen>

▸<total>

▸<totalUnconfirmed>

▸<aggregate>

▸<goal>

▸<missedPen>

▸<card>

- ▸<substitute>
- ▸<VAR>
- ▸<penaltyShootout>
- ▸<penaltyShot>
- ▸<lineUp>
- ▸<player>
- ▸<stat>
- ▸<teamOfficial>
- ▸<teamStats>
- ▸<stat>
- ▸<kit>
- ▸<matchDetailsExtra>
- ▸<attendance>
- ▸<awayAttendance>
- ▸<matchOfficials>(XML only)
- ▸<matchOfficial>

How Are VAR Events Reported?

An update to the feed (released 3 June 2019) means that when a VAR (Video Assistant Referee) review takes place our analysts will collect the type of event being reviewed, the team and player involved, and the decision of the review (outcome) which also describes what happens next.

Outcomes

The following table gives a breakdown of possible outcomes from a VAR review, according to the VAR event type/reason and the referee decision.

The outcomes appear in the `VAR` element of the feed response. For a list of all VAR-related attributes that may appear during/after a match, see the VAR elements/attributes and Feed Output sections in this guide.

- `type` (`VAR` element) - describes the original event/decision that is the reason for the VAR review
- `outcome` (`VAR` element) - the outcome of the VAR review; the value is the event that stands in the match
- `decision` (`VAR` element) - the referee decision following the VAR review, in relation to the original decision: `Cancelled` (original event cancelled, the 'outcome' value gives the event that will stand); `Confirmed` (original event confirmed and stands, as per the 'outcome' value)

VAR event varReason **value,** matchDetails **node /type value,** VAR **node**	**Referee decision** decision **attribute value**	**Outcome** outcome **attribute value**	**Explanation**
`Goal awarded` *	`Confirmed`	`Goal`	*Original decision **is not** changed after review*
`Goal awarded` *	`Cancelled`	`No goal`	*Original decision **is changed** after review*
`Goal not awarded` *	`Confirmed`	`No goal`	*Original decision **is not** changed after review*
`Goal not awarded` *	`Cancelled`	`Goal`	*Original decision **is changed** after review*
`Penalty awarded`	`Confirmed`	`Penalty`	*Original decision **is not** changed after review*
`Penalty awarded`	`Cancelled`	`No penalty`	*Original decision **is changed** after review*
`Penalty not awarded`	`Confirmed`	`No penalty`	*Original decision **is not** changed after review*
`Penalty not awarded`	`Cancelled`	`Penalty`	*Original decision **is changed** after review*
`Red card given`	`Confirmed`	`Red card`	*Original decision **is not** changed after review*
`Red card given`	`Cancelled`	`No red card`	Original decision **is changed** after review
`Card upgrade`	`Confirmed`	`No change`	*Original decision **is not changed** after review*
`Card upgrade`	`Cancelled`	`Card upgraded`	*Original decision **is** changed after review*
`Mistaken identity`	`Confirmed`	`Player originally cautioned/ sent off confirmed`	*Original decision **is not** changed after review*
`Mistaken identity`	`Cancelled`	`Ref cautions/ sends off a different player`	*Original decision **is changed** after review*
`Other`	`Cancelled`	n/a	*Original decision **is not** changed after review* **Note:** Added 22 July 2019

VAR event varReason **value,** matchDetails **node** /type **value,** VAR **node**	**Referee decision** decision **attribute value**	**Outcome** outcome **attribute value**	**Explanation**
`Other`	`Confirmed`	n/a	*Original decision **is changed** after review* **Note:** Added 22 July 2019

VAR Elements/Attributes

The following table describes the different ways that we use VAR attributes. For more information, see the relevant elments in the Feed Output section.

DESCRIPTION
VAR review flag • These attributes show that: 1) there is a delay; 2) the reason (incident type) requiring the review; 3) the relevant t • They are only shown in feed when a VAR review is taking place; the attributes and values disappear from the fee
VAR information for goal events • These attributes show that: 1) a goal has been awarded following a review, and; 2) whether the goal was: a) originally awarded and confirmed after review, or; b) only awarded following a review • Note: The player attached to a VAR Goal event may not always be the scorer of the goal under review. In some c
All VAR events in a game • The `VAR` element will appear in any game where there has been a VAR review. • It will list all VAR events in a game, the player and team involved, the type of event reviewed, whether the original • This list will update during a game as more events are reviewed.

Statistic Types

Statistics are delivered under the `stats` object and split into the following elements (where available):

- `team` (contestant)
- `player` (in the team)
- `combinedTeamStats` (a total sum of statistics for each team)

Within these elements, each statistic is shown as a `stat` and its `type` and `value` will appear under it, if it is available. For example, the `yellowCard` stat type would only appear if a yellow card was awarded to a contestant (team) or player (it would not appear if a yellow card was not awarded).

> **Note:**
> • **shotOffTarget** = includes all the miss and post events/stat types. Calculated as `attOboxMiss` + `attIboxMiss` + `attOboxPost` + `attIboxPost` (= `shotOffTarget`)
> • **total number of missed shots** = shots that went wide or over. Calculated as either a) `attIboxMiss` + `attOboxMiss`, OR b) `shotOffTarget` – `postScoringAtt`

Click on the buttons below to see the complete lists of contestant (team) and player statistics.

Contestant/Team Stats

For these stats, the value will be an *integer* with the exception of `formationUsed` which can have an *integer*, or a *string* such as `Unknown`.

▸**Contestant/Team stats - names and descriptions**

Player Stats

For these stats, the value will be an *integer* with the exception of `formationPlace` which can have a *positive integer*, or `0` or a *string* such as `Unknown`.

▸**Player stats - names and descriptions**

Check The Table Below To See Which Stats Are Available In Each Tier

▸**Stats under each Tier**